SCV SECURITY

AUDIT REPORT

Phoenix Directive

Terra Core (Cosmos-SDK)

Prepared by SCV-Security

On 27th May 2025

Table of Contents

Table of Contents **2**
Introduction **3**
Scope Functionality 3
Submitted Codebase 3
Submitted Codebase - Revisions 3
Methodologies 4
Code Criteria 5
Findings Summary **6**
Audit Observations **7**
1. Fee shares can be forcefully registered 7
Findings Technical Details **8**
1. SetBeforeSendHook can be weaponized to cause a denial of service 8
2. Users can send tokens to blocked module addresses 9
4. SendCoinsFromModuleToModule does not implement tokenfactory hooks 10
5. Hooks are not called before minting new tokens 11
6. Incorrect events emitted 12
7. ErrFeeShareDisabled error code is incorrectly registered as 1 13
8. Vulnerable wasmd version used 14
9. Misleading withdrawerAddrs variable 15
10. Unused parameter 16
Document Control **17**
Appendices **18**
A. Appendix - Risk assessment methodology 18
B. Appendix - Report Disclaimer 19

Introduction

SCV has been engaged by Phoenix Directive to conduct a comprehensive security review with the goal of identifying potential security threats and vulnerabilities within the codebase. The purpose of this audit is to evaluate the security posture of the codebase and provide actionable recommendations to mitigate any identified risks. This report presents an overview of the findings from our security audit, outlining areas of concern and proposing effective measures to enhance the codebase's security.

Scope Functionality

Terra is a public, open-source, decentralized blockchain hosting a vibrant ecosystem of decentralized applications and top-tier developer tools. The Terra Core is the full-node software implementation of the Terra blockchain, powered by the Cosmos SDK and Tendermint BFT consensus.

Submitted Codebase

Terra Core v2.13 – Cosmos SDK	
Repository	https://github.com/phoenix-directive/core/tree/release/v2.13
Commit	`61f51b4fe354746f7477e00b7b68c2c4f382f7fb`
Branch	`release/v2.13`

Submitted Codebase – Revisions

Terra Core v2.17.0 – Cosmos SDK	
Repository	https://github.com/phoenix-directive/core/releases/tag/v2.17.0
Commit	`62b2d6925831e95656e1f7de8719bde1c5b8daaf`
Branch	`release/v2.17.0`

Methodologies

SCV performs a combination of automated and manual security testing based on the scope of testing. The testing performed is based on the extensive experience and knowledge of the auditor to provide the greatest coverage and value to Phoenix Directive. Testing includes, but is not limited to, the following:

- Understanding the application and its functionality purpose.
- Deploying SCV in-house tooling to automate dependency analysis and static code review.
- Analyse each line of the code base and inspect application security perimeter.
- Review underlying infrastructure technologies and supply chain security posture.

Code Criteria

This section provides an evaluation of specific criteria aspects as described below:

- **Documentation:** Evaluating the presence and comprehensiveness of publicly available or provided explanatory information, diagram flowcharts, comments, and supporting documents to enhance code understanding.
- **Coverage:** Evaluating whether the code adequately addresses all necessary cases and scenarios, ensuring that the intended functionality or requirements are sufficiently covered.
- **Readability:** Assessing how easily the code can be understood and maintained, considering factors such as code structure, naming conventions, and overall organisation.
- **Complexity:** Evaluating the complexity of the code, including factors such as, number of lines, conditional statements, and nested structures.

The status of each criteria is categorised as either **SUFFICIENT** or **NOT-SUFFICIENT** based on the audit assessment. This categorisation provides insights to identify areas that may require further attention and improvement.

Criteria	Status	Notes
Documentation	**SUFFICIENT**	N/A
Coverage	**SUFFICIENT**	N/A
Readability	**SUFFICIENT**	N/A
Complexity	**SUFFICIENT**	N/A

Findings Summary

Summary Title	Risk Impact	Status
`SetBeforeSendHook` can be weaponized to cause a denial of service	CRITICAL	RESOLVED
Users can send tokens to blocked module addresses	SEVERE	RESOLVED
`SendCoinsFromModuleToModule` does not implement tokenfactory hooks	MODERATE	RESOLVED
Hooks are not called before minting new tokens	MODERATE	RESOLVED
Incorrect events emitted	LOW	RESOLVED
`ErrFeeShareDisabled` error code is incorrectly registered as `1`	LOW	RESOLVED
Vulnerable `wasmd` version used	LOW	RESOLVED
Misleading `withdrawerAddrs` variable	INFO	RESOLVED
Unused parameter	INFO	RESOLVED

Audit Observations

The audit observations section is intended to present potential findings that are related to the underlying design of the protocol and would require underlying design changes to remediate that may change the overall functioning of the protocol. SCV asks that the client formulate responses to add context to validate or invalidate the following concerns.

1. Fee shares can be forcefully registered

The `RegisterFeeShare` function in `x/feeshare/keeper/msg_server.go:129-141` allows anyone to register a fee share for contracts created from the factory (determined by the `GetIfContractWasCreatedFromFactory` function). This would cause the contracts to receive the fees when users interact with them, as the withdrawal address is automatically set to the contract address.

The issue is that some contracts may not intend to receive fees, which may cause the funds to be stuck. If the contract has renounced its admin, it cannot be migrated to withdraw the fees. Additionally, if the contract removes the fee share registration with `MsgCancelFeeShare`, users could still register the fee share again on behalf of the contract.

Potential Remediation:

Consider implementing a functionality for governance to blacklist certain contracts from being registered. This should be permissioned and callable by the `x/gov` module address only.

Findings Technical Details

1. `SetBeforeSendHook` can be weaponized to cause a denial of service

RISK IMPACT: CRITICAL	STATUS: RESOLVED

Description

The `SetBeforeSendHook` feature allows a token creator to set a CosmWasm contract that will be executed when a token transfer occurs. This allows the creator to control the token flows, such as preventing a specific recipient from receiving the funds.

However, this opens up a potential denial of service issue if module accounts own the tokens and want to transfer them during ABCI calls. The token creator can weaponize the `SetBeforeSendHook` feature to force the module accounts to fail to transfer the funds. This would cause a chain halt as the error occurs within ABCI instances, causing a denial of service.

Recommendation

Consider modifying the `SetBeforeSendHook` feature to be permissioned and only allow whitelisted code IDs and token creators to use this feature. An example can be found here.

2. Users can send tokens to blocked module addresses

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

The chain implements a list of blocked module addresses that cannot receive funds externally. For example, if users try to send funds to the "fee_collector" module address, the transaction will fail with an error message stating the module is not allowed to receive funds.

However, two instances allow users to bypass this restriction:

1. The `PerformMint` function in `x/tokenfactory/bindings/message_plugin.go:147` does not validate that the `MintToAddress` is not a blocked address.
2. The `forceTransfer` function in `x/tokenfactory/keeper/bankactions.go:106` does not validate that the `toAddr` is not a blocked address.

This may cause unintended consequences as these modules may not intend to receive tokenfactory denoms. For instance, ASA-2024-003 mentions a possible chain halt scenario due to missing blocked address validation.

Recommendation

Consider applying the following recommendations:

1. Remove the `SendCoins` call and pass `MintToAddress` to the `NewMsgMintTo` function (`x/tokenfactory/types/msgs.go:69`), replacing the old `NewMsgMint`. The `SendCoinsFromModuleToAccount` function will automatically verify that the recipient address is not blocked before transferring funds.
2. Perform the same validation as `fromAddr` (`x/tokenfactory/keeper/bankactions.go:90-99`) to ensure `toAddr` is not a module account.

4. SendCoinsFromModuleToModule does not implement tokenfactory hooks

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

The x/bank module defines a custom bank keeper that supports executing hooks defined in the x/tokenfactory module. However, in x/bank/keeper/keeper.go:99-110, the SendCoinsFromModuleToModule function does not call the BlockBeforeSend and TrackBeforeSend hooks when transferring the funds.

Recommendation

Consider calling the BlockBeforeSend and TrackBeforeSend functions so the hooks logic is enforced in SendCoinsFromModuleToModule.

5. Hooks are not called before minting new tokens

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

Before a token transfer occurs, the `BlockBeforeSend` and `TrackBeforeSend` hooks must be called to enforce the `x/tokenfactory` hooks logic properly. For example, before burning the tokens, the hooks are called in `x/tokenfactory/keeper/bankactions.go:63-67`.

The issue is that the hooks are not called beforehand before minting new tokens in `x/tokenfactory/keeper/bankactions.go:23`. This means that if the hook should halt the mint operation or implement a custom balance tracking, it will not work correctly.

Recommendation

Consider calling the `BlockBeforeSend` and `TrackBeforeSend` hooks before minting new tokens.

6. Incorrect events emitted

RISK IMPACT: LOW	STATUS: RESOLVED

Description

The “mint_to_address” and “burn_from_address” attributes are emitted as the sender’s address in `x/tokenfactory/keeper/msg_server.go:92` and `x/tokenfactory/keeper/msg_server.go:134`. This is incorrect because it should emit the `msg.MintToAddress` and `msg.BurnFromAddress` addresses.

Recommendation

Consider updating the attributes to emit correct values.

7. `ErrFeeShareDisabled` error code is incorrectly registered as 1

RISK IMPACT: LOW	STATUS: RESOLVED

Description

The `ErrFeeShareDisabled` error is registered with the code value `1` in `x/feeshare/types/errors.go:8`. This is incorrect because, based on the Cosmos SDK documentation, a code value of 1 is reserved for internal errors, and other errors should be registered with a value greater than one.

Recommendation

Consider updating the error code to a different value.

8. Vulnerable `wasmd` version used

RISK IMPACT: LOW	STATUS: RESOLVED

Description

The chain uses an outdated version of `wasmd`, which is `v0.46.0`. This is problematic because this version is vulnerable to CWA-2024-003 and CWA-2024-009, potentially exposing a denial of service issue.

Recommendation

Consider updating the `wasmd` version to `v0.53.2`.

9. Misleading `withdrawerAddrs` variable

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `x/feeshare/post/post.go:90`, the `withdrawerAddrs` variable is iterated to transfer fees to the withdrawer addresses. However, the `withdrawerAddrs` variable is re-declared in the loop as the withdrawer's individual address, decreasing code readability and maintainability.

```
for _, withdrawerAddrs := range withdrawerAddrs { … }
```

Recommendation

Consider updating the variable to `addr`.

```
for _, addr := range withdrawerAddrs { … }
```

10. Unused parameter

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

The `AfterExecuteContract` function in `x/wasm/keeper/keeper.go:75` implements a `types.MsgExecuteContractResponse` parameter, but it is not used.

Recommendation

Consider removing the unused parameter.

Document Control

Version	Date	Notes
-	16th January 2025	Security audit commencement date.
0.1	3rd February 2025	Initial report with identified findings delivered.
0.5	15th Feb - 20 May 2025	Fixes remediations implemented and reviewed.
1.0	27th May 2025	Audit completed, final report delivered.

Appendices

A. Appendix – Risk assessment methodology

SCV-Security employs a risk assessment methodology to evaluate vulnerabilities and identified issues. This approach involves the analysis of both the LIKELIHOOD of a security incident occurring and the potential IMPACT if such an incident were to happen. For each vulnerability, SCV-Security calculates a risk level on a scale of 5 to 1, where 5 denotes the highest likelihood or impact. Consequently, an overall risk level is derived from combining these two factors, resulting in a value from 10 to 1, with 10 signifying the most elevated level of security risk

Risk Level	Range
CRITICAL	10
SEVERE	From 9 to 8
MODERATE	From 7 to 6
LOW	From 5 to 4
INFORMATIONAL	From 3 to 1

LIKELIHOOD and **IMPACT** would be individually assessed based on the below:

Rate	LIKELIHOOD	IMPACT
5	**Extremely Likely**	Could result in severe and irreparable consequences.
4	**Likely**	May lead to substantial impact or loss.
3	**Possible**	Could cause partial impact or loss on a wide scale.
2	**Unlikely**	Might cause temporary disruptions or losses.
1	**Rare**	Could have minimal or negligible impact.

B. Appendix – Report Disclaimer

This report should not be regarded as an "endorsement" or "disapproval" of any specific project or team. These reports do not indicate the economics or value of any "product" or "asset" created by a team or project that engages SCV-Security for a security review. The audit report does not make any statements or warranties about the code's utility, safety, suitability of the business model, regulatory compliance of the business model, or any other claims regarding the fitness of the implementation for its purpose or its bug-free status. The audit documentation is intended for discussion purposes only. The content of this audit report is provided "as is," without representations and warranties of any kind, and SCV-Security disclaims any liability for damages arising from or in connection with this audit report. Copyright of this report remains with SCV-Security.

THANK YOU FOR CHOOSING



SCV SECURITY

scv.services

contact@scv.services